

Document number: P0037R7

Date: 2019-06-17

Reply-to: John McFarlane, wg21@john.mcfarlane.name

Audience: SG6, LEWGI

Fixed-Point Real Numbers

Introduction

This proposal introduces a system for performing fixed-point arithmetic using integral types.

Contents

- [Motivation](#)
- [Impact On the Standard](#)
- [Design Decisions](#)
 - [Class Template](#)
 - [Rep](#) Type Template Parameter
 - [Scale](#) Type Template Parameters
 - [Exponent](#) and [Radix](#) Non-Type Template Parameters
 - [Requirements on Rep](#)
 - [Conversion](#)
 - [Access to Rep](#) Value
 - [Class Template Deduction](#)
 - [Operators](#)
 - [Division Operator](#)
 - [Custom Division](#)
 - [Alternative Types for Rep](#)
 - [Required Specializations](#)
 - [Example](#)
- [Technical Specification](#)
 - [Header <scaled_integer> Synopsis](#)
 - [scaled_integer<>](#) Class Template
- [Open Issues](#)
 - [Library Support](#)
 - [Extended Comparison Range](#)
 - [Allow Binary Operations if Radixes are Different](#)
- [Prior Art](#)
 - [N1169](#)
 - [P0106](#)
 - [Ada Language Support](#)
- [Acknowledgements](#)
- [Revisions](#)
- [Appendix 1: Reference Implementation](#)
- [Appendix 2: Performance](#)
 - [Types](#)
 - [Basic Arithmetic](#)
 - [3-Dimensional Magnitude Squared](#)
 - [Circle Intersection](#)

Motivation

Floating-point types are an exceedingly versatile and widely supported method of expressing real numbers on modern architectures.

However, there are certain situations where fixed-point arithmetic is preferable:

- Some systems lack native floating-point registers and must emulate them in software;
- many others are capable of performing some or all operations more efficiently using integer arithmetic;
- certain applications can suffer from the variability in precision which comes from a dynamic radix point [\[pathengine\]](#);
- binary arithmetic operations including `+`, `-`, `*` and `/` result in underflow, i.e. loss of lower-precision bits;
- in situations where a variable exponent is not desired, it takes valuable space away from the significand and reduces precision and
- not all hardware and compilers produce exactly the same results, leading to non-deterministic results.

Integer types provide the basis for an efficient, lossless representation of binary fixed-point real numbers. However, laborious, error-prone steps are required to normalize the results of certain operations and to convert to and from fixed-point types.

A set of tools for defining and manipulating fixed-point types is proposed. These tools are designed to make work easier for those who traditionally use integers to perform low-level, high-performance fixed-point computation. They are composable such that a wide range of trade-offs between speed, accuracy and safety are supported.

Impact On the Standard

This proposal is a pure library extension. It does not require changes to any standard classes or functions. It adds several new class and function templates to new header file, `<scaled_integer>`. Some optional deduction guides, member functions and operator overloads rely on types proposed in [\[P0828\]](#) and [\[P1050\]](#).

Design Decisions

The design is driven by the following aims in roughly descending order:

1. to automate the task of performing low-level fixed-point arithmetic using integer types — including non-standard fundamental integers or classes;
2. to minimise precision loss due to underflow;
3. to treat fixed-point as a super-set of integer such that a fixed-point type with an exponent of zero can provide a drop-in replacement for its underlying integer type;
4. to avoid incurring expense — including compilation time — for unused features and
5. to facilitate a style of code that is intuitive to anyone who is comfortable with integer and floating-point arithmetic.

More generally, the aim of this proposal is to contain within a single API all the tools necessary to perform fixed-point arithmetic. The design facilitates a wide range of competing compile-time strategies for avoiding overflow and precision loss, but implements only the simplest by default. Similarly, orthogonal concerns such as run-time overflow detection and rounding modes are deferred to the underlying integer types used as storage.

Class Template

Fixed-point numbers are specializations of

```
template <class Rep = int, class Scale = power<>>
class scaled_integer;
```

where `power` is a tag type declared as

```
template <int Exponent = 0, int Radix = 2>
struct power;
```

and where the template parameters are described as follows.

Rep Type Template Parameter

This parameter indicates the integral type used as storage. Fundamental integral types other than `bool` are ideal choices but any suitably integer-like type can be used.

Other than scale, the characteristics of `scaled_integer<Rep>` are the characteristics of `Rep` including:

- signedness;

- number of digits;
- behavior of operators and
- alignment.

Scale Type Template Parameter

`Scale` is a tag type used to specify the type of static scaling applied to the integer in order to convert it between its underlying value and the semantic value of the `scaled_integer` type. Future specification of the relationship between `Scale` and `Rep` could be used to allow user-defined alternatives to `power`. An example of an existing type which might replace `power` is `ratio`.

Exponent and Radix Non-Type Template Parameters

The radix (or base) of a fixed-point type is typically two to denote scaling by powers of two. In financial applications, accurate representation of decimal fractions requires a radix of ten. Thus while `Radix` can be any number greater than one, `2` is the default.

The exponent of a fixed-point type is the equivalent of the exponent field in a floating-point type and shifts the stored value by the requisite number of digits necessary to produce the desired range. The default value of `Exponent` is zero, giving `scaled_integer<T>` the same range as `T`. By far the most common use of fixed-point is to store values with fractional digits. Thus, the exponent is typically a negative value.

The resolution of an instantiation of `scaled_integer` is

```
pow(Radix, Exponent)
```

and the minimum and maximum values are

```
std::numeric_limits<Rep>::min() * pow(Radix, Exponent)
```

and

```
std::numeric_limits<Rep>::max() * pow(Radix, Exponent)
```

respectively.

Any usage that results in values of `Exponent` which lie outside the range, $(\text{INT_MIN} / 2, \text{INT_MAX} / 2)$, may result in undefined behavior and/or overflow or underflow. This range of exponent values is far in excess of the largest built-in floating-point type and should be adequate for all intents and purposes.

Requirements on Rep

First and foremost, `Rep` is required to be an arithmetic component as discussed in [\[P0554\]](#).

Semantically `Rep` is a fundamental integral type or a class emulating parts of such a type necessary to invoke the equivalent `scaled_integer` operation. For example, `scaled_integer<Rep>`'s binary `operator+` requires that `Rep` similarly has an arithmetic binary `operator+`. To be convertible to/from other arithmetic types, `Rep` must support the same conversion.

Additionally, `Rep` must be scalable, which is to say it must support arithmetic left-shift, multiply and division. This is necessary — not only for conversion to and from other arithmetic types but also — in order to normalize the operands in operations such as addition and comparison.

Finally, `Rep` should provide specializations for the customisation points laid down in [Required Specializations](#).

Conversion

While effort is made to ensure that significant digits are not lost during conversion, no effort is made to avoid rounding errors. Whatever would happen when converting to and from `Rep` largely applies to `scaled_integer` objects also. For example:

```
scaled_integer<int, power<-1>>{.499}==0
```

...equates to `true` and is considered an acceptable rounding error.

Access to Rep Value

It is sometimes necessary to read from and write to the `Rep` value contained in a `scaled_integer<Rep>` object. This is supported through numeric traits, `to_rep` and `from_rep` respectively. These traits are described in paper, [P0675].

```
constexpr auto a = from_rep<scaled_integer<int, power<-8>>>{}>>(320);
static_assert(a == 1.25);

constexpr auto b = to_rep(a);
static_assert(b == 320);    // 1.25*(1<<8)
```

Class Template Deduction

The type of a `scaled_integer` object can be deduced by an integer initializer:

```
auto a = scaled_integer(0ul);
static_assert(is_same_v<decltype(a), scaled_integer<unsigned long>>);
```

It can also be deduced with an integral constant of type `constant` (described in [P0827]):

```
constexpr auto b = scaled_integer(constant<0xFF00000000L>{});
static_assert(is_same_v<decltype(b), const scaled_integer<int, power<32>>>);
static_assert(to_rep(b) == 0xFF);
```

For `Exponent`, the highest value which does not incur data loss is used. This minimizes the required range of the underlying integer value which reduces the likelihood of out-of-range errors during arithmetic operations. For `Rep`, a fundamental integer type of `int` width is preferred unless a wider type is required.

Operators

Any arithmetic, comparison, logic and bitwise operators that might be applied to integer types can also be applied to fixed-point types. A guiding principle of operator overloads is that they perform as little run-time computation as is practically possible.

With the exception of shift and comparison operators, binary operators can take any combination of:

- one or two fixed-point arguments and
- zero or one arguments of any arithmetic type, i.e. a type for which `numeric_limits` is specialized.

Assuming a binary operation, `@`, in the form

```
auto R = S @ T;
```

where `S` is of type `scaled_integer<RepS, power<ExponentS, 2>>` and `T` is a numeric type — possibly another `scaled_integer` instantiation — then result, `R`, of the operation is determined as follows.

1. If `T` is a floating-point type, `Float`, then `S` is cast to `Float` and a floating-point operation takes place, e.g.:

```
auto a = scaled_integer<long long>(3) + 4.f;
static_assert(is_same_v<decltype(a), decltype(3.f + 4.f)>);
```

2. If `T` is a `constant` of integer type, `Integer`, then:

- a. If the operator is bitwise left shift (`<<`), then the result is `scaled_integer<RepS, power<ExponentS+T::value>>` with the shift operator applied.
- b. If the operator is bitwise right shift (`>>`), then the result is `scaled_integer<RepS, power<ExponentS-T::value>>` with the shift operator applied.
- c. Otherwise, `T` is converted to `scaled_integer(T{})`, e.g.

```
auto b = scaled_integer(200U) - constant<100L>{};
static_assert(is_same_v<
    decltype(b),
    decltype(scaled_integer<unsigned>(200) - scaled_integer<int>(100))>);
```

and proceeding rule #4 subsequently applies.

3. If `T` is an integer type, `Integer`, then:

a. If the operator is bitwise shift (`<<` or `>>`), then the result is type `S` with the shift operator applied.

b. Otherwise, `T` is cast to `scaled_integer<Integer>`, e.g.

```
auto c = scaled_integer<>(5) * 6ul;
static_assert(is_same_v<
    decltype(c),
    decltype(scaled_integer<>(5) * scaled_integer<unsigned long>(6))>);
```

and proceeding rule #4 subsequently applies.

4. If `T` is type, `scaled_integer<RepT, power<ExponentT, 2>>`, then:

a. If the operator is multiplication (`*`), then the result is `scaled_integer<decltype(RepS*RepT), power<ExponentS+ExponentT>>`, e.g.:

```
constexpr auto d = scaled_integer<uint8_t, power<-7>>{1.25}
    * scaled_integer<uint8_t, power<-3>>{8};
static_assert(is_same_v<decltype(d), const scaled_integer<int, power<-10>>>);
static_assert(d == 10);
```

b. If the operator is division (`/`), then the result is `scaled_integer<decltype(RepS/RepT), power<ExponentS-ExponentT>>`, e.g.:

```
constexpr auto e = scaled_integer<short, power<-5>>{1.5} / scaled_integer<short, power<-3>>{2.5};
static_assert(is_same_v<decltype(e), const scaled_integer<int, power<-2>>>);
static_assert(e == .5);
```

c. If the operator is modulo (`%`), then the result is `scaled_integer<decltype(RepS%RepT), power<ExponentS>>`, e.g.:

```
constexpr auto f = scaled_integer<short, power<-5>>{1.5} % scaled_integer<short, power<-3>>{2.5};
static_assert(is_same_v<decltype(f), const scaled_integer<int, power<-5>>>);
static_assert(f == .25);
```

d. If the operator is addition (`+`) or subtraction (`-`), then the operand with the greater exponent is converted such that its exponent matches the other operands' exponent. Then the result is `scaled_integer<decltype(RepS@RepT), power<min(ExponentS, ExponentT)>>`, eg.:

```
constexpr auto g = scaled_integer<int8_t, power<-2>>{12.5} - scaled_integer<short>{8};
static_assert(is_same_v<decltype(g), const scaled_integer<int, power<-2>>>);
static_assert(g == 4.5);
```

e. If the operator is comparison (`==`, `!=`, `<`, `>`, `<=` or `>=`), then the operand with the greater exponent is converted such that its exponent matches the other operands' exponent. Then the result is `decltype(RepS@RepT)`, eg.:

```
constexpr auto h = scaled_integer<int8_t, power<-2>>{12.5} <= scaled_integer<short>{8};
static_assert(is_same_v<decltype(h), const bool>);
static_assert(h == false);
```

(See section, [Extended Comparison Range](#), for additional details.)

f. If the operator is bitwise or (`|`) or xor (`^`) then the same rules as addition (`+`) are applied.

g. If the operator is bitwise and (`&`) then the same rules as bitwise or (`|`) are applied except that the greater — not less — exponent is preferred.

Some details have been left out for brevity. Unary operators are supported. Some minor variations occur when `S` is not `scaled_integer` and `T` is `scaled_integer`. Rules for bit-shifting values where `Radix!=2` do not necessarily involve a different result type. Other binary operations involving different radices produce a return type which is optimized to contain the

precise result with the minimum value stored in `Rep` and the minimum viable value of `Radix`.

The complete set of rules may appear to be large and complex. However, this mostly reflects the existing complexity in the behavior of arithmetic types. Relatively few design principles govern these rules:

1. A `scaled_integer<T>` should follow the same behavior as `T` to the greatest extent practical, reflecting the facts that: a) all integers are fixed-point — rather than floating-point types and b) integer arithmetic generally provides the best efficiency and performance characteristics.
2. In situations where a trade-off between overflow and underflow must be made, the design guards against underflow. This follows from principle #1. Far more operations can cause overflow and users are generally more wary of it. And detection/handling of overflow is an orthogonal concern which is best implemented using a custom numeric type such as the `safe_integer` and `elastic_integer` types discussed in [P0554].

Division Operator

The behavior of the division operator, `operator/`, poses a dilemma which has proven contentious. Following an impromptu review of P0037 by SG6 in San Diego Davis Herring volunteered to write a paper which explores two competing strategies as identified by SG6 [P1368]. The two strategies identified are:

1. 'Quasi-exact' is the strategy pursued by P0106 and perceived as being most like floating-point and therefore least surprising.
2. The strategy currently proposed for P0037 which follows the behavior of integer division and therefore maximizes efficiency, control, precision and consistency with the other operators.

Unfortunately, fixed-point is commonly seen as a replacement for floating-point — rather than an extension to integer — arithmetic. There is a desire to write generic code which can accept both floating-point and fixed-point types and which involves division operations. However, there are problems with this aim which are not necessarily solved by using 'quasi-exact' division. Conversely, it is feasible to write generic code in which fixed-point and integer types can be interchanged. 'Quasi-exact' is a poor fit for such code. Finally, generic code which can be successfully instantiated with all three is possible but must be written with extra care where division is concerned.

Some observations which may help back up these claims are as follows.

'Quasi-exact' quotients are sensitive to operand types

Consider the following examples using P0106 types:

```
auto q = negatable{1} / negatable{3L} # example 1
```

In the above example, the type of the dividend and divisor are deduced from the initializer. On systems with 32-bit `long` integers, the type of the divisor will be deduced as `negatable<31, 0>` and on systems with 64-bit `long` integers, the type of the divisor will be deduced as `negatable<63, 0>`. This will in turn affect the number of fractional digits of the quotient which will in turn affect the resultant value.

Certainly, there are ways in which this situation can be avoided but in general, making the result sensitive to the range of the divisor may have drawbacks in some situations.

```
auto q = negatable<1, 0>{1} / negatable<2, 0>{3} # example 2
```

In this second example, the minimum width is chosen for the operands. The result is now consistent but is not exact at all, having only two fractional digits.

'Quasi-exact' results are inexact

By trying to emulate floating-point division, precision loss is considered acceptable. In contrast, integer division is lossless — provided that the remainder is taken into account. Consider the example from the [Operators](#) section.

```
// quotient is scaled_integer<short, power<-2>>{.5}
constexpr auto e = scaled_integer<short, power<-5>>{1.5} / scaled_integer<short, power<-3>>{2.5};

// remainder is scaled_integer<short, power<-5>>{.25}
constexpr auto f = scaled_integer<short, power<-5>>{1.5} % scaled_integer<short, power<-3>>{2.5};

// dividend is scaled_integer<int, power<-5>>{1.5}
```

```
constexpr auto dividend = scaled_integer<int, power<-3>>{2.5} * e + f;
```

The initial dividend is retrieved by working back from the quotient, the remainder and the divisor. This is guaranteed by the underlying integer type. Indeed, an optimizing compiler will recognize that the input and output dividend are the same and elide them. (example)

'Quasi-exact' division is slow

The above example requires no shift operations. P1368 [P1368] speculates that for some `Rep` types, shift operations may be elided. This is a valid point. However, nothing about providing a thin abstraction precludes such elision. And at this level the bare amount of scaling can be performed regardless of `Rep` type.

'Quasi-exact' division is rigid

What 'quasi-exact' division really delivers is a reliable way to get *some* precision. The fact that it takes a best guess is evidence of a lack of control afforded the user. And the user is badly served when this guess is wrong. The ideal API should provide this best guess as a default only and allow the user to override it to choose the precision they need. That is the topic of the next section.

Custom Division

The `scaled_integer` division operator, `/`, performs the least work possible and, combined with the modulo operator, `%`, produces lossless results. However, it behaves very differently from floating-point division and is likely to be a source of surprises for some users.

In particular, the choice of quotient type can have a dramatic effect on precision. If, for example, the dividend and divisor have the same `Exponent` and `Radix`, then the quotient's `Exponent` will be zero and all fractional digits will be dropped. In contrast to floating-point division, the choice of `Exponent` cannot be tailored to the result.

For this reason, the `fractional` type [P1050] is provided in order to facilitate two important use cases.

Firstly a 'sane default' result type can be calculated automatically in line with the formula detailed in P0106 [P0106]. Here, a deduction guide does the work of determining that a division involving a dividend with 31 integer digits should result in a quotient with 31 fractional digits:

```
constexpr auto i = scaled_integer{fractional{1, 3}};  
static_assert(i == 0.33333333022892475128173828125L);  
static_assert(is_same_v<decltype(i), const scaled_integer<int64_t, power<-31>>>);
```

This suffers from the problem (discussed above) that number of fractional digits is sensitive to the width of the divisor. However, the `scaled_integer` type does not attempt to abstract away the integers with which it represents values. This should make such portability issues more apparent. And in the above style, retention of fractional values using the `fractional` type becomes encouraged.

Alternatively, the user can forgo CTAD and choose the template parameters explicitly:

```
constexpr auto j = scaled_integer<int, power<-16>>{fractional{1, 3}};  
static_assert(j == 0.3333282470703125);  
static_assert(is_same_v<decltype(j), const scaled_integer<int, power<-16>>>);
```

This usage avoids all ambiguity and provides complete control over the precision expressed in the quotient.

Alternative Types for `Rep`

Using built-in integral types as the default underlying representation minimizes certain costs:

- many fixed-point operations are as efficient as their integral equivalents;
- compile-time complexity is kept relatively low and
- the behavior of fixed-point types should cause few surprises.

However, this choice also brings with it many of the deficiencies of built-in types. For example:

- the typical rounding behavior is distinct for:
 - conversion from floating-point types;

- right shift and
- divide operations;
- all of these rounding behaviors cause drift and propagate error;
- overflow, underflow and flush are handled silently with wrap-around or undefined behavior;
- divide-by-zero similarly results in undefined behavior and
- the range of values is limited by the largest type: `long long int`.

The effort involved in addressing these deficiencies is non-trivial and on-going (for example [P0105]). As solutions are made available, it should become easier to define custom integral types which address concerns surrounding robustness and correctness. How to combine such numeric types is the topic of [P0554], **Composition of Arithmetic Types**.

Of particular note is the `elastic_integer` type detailed in [P0828]. When used in combination with `scaled_integer`, the resultant composite type is able to avoid a large proportion of the out-of-range errors associated with fixed-point arithmetic while avoiding expensive run-time overflow checks.

Required Specializations

For a type to be suitable as parameter, `Rep`, of `scaled_integer`, it must meet the following requirements:

- it must have specialized the following existing standard library types:
 - `numeric_limits`
 - `make_signed` and `make_unsigned`
- it must have specialized the following proposed standard library types as described in [P0675]:
 - `num_digits` and `set_num_digits`,
 - `to_rep`, `from_rep` and `from_value`.

Note that `make_signed` and `make_unsigned` cannot be specialized for custom types. Unless this rule can be relaxed, some equivalent mechanism must be introduced in order for custom types to be used with `scaled_integer<>`. One possibility is the addition of `numeric_limits<>::signed` and `numeric_limits<>::unsigned` type aliases.

Example

The following function, `magnitude`, calculates the magnitude of a 3-dimensional vector.

```
template<class Fp>
constexpr auto magnitude(Fp x, Fp y, Fp z)
{
    return sqrt(x*x+y*y+z*z);
}
```

And here is a call to `magnitude`.

```
auto m = magnitude(
    scaled_integer<uint16_t, power<-12>>(1),
    scaled_integer<uint16_t, power<-12>>(4),
    scaled_integer<uint16_t, power<-12>>(9));
// m === scaled_integer<uint32_t, power<-24>>{9.8994948863983154}
```

Technical Specification

Header `<scaled_integer>` Synopsis

```
namespace std {
    template <int Exponent, int Radix> class power;

    template <class Rep, class Scale> class scaled_integer;

    // for each unary arithmetic, comparison, logic and bitwise operator, @
    template <class RhsRep, int RhsExponent, int RhsRadix>
        constexpr auto operator@(
```

```

    const scaled_integer<RhsRep, power<RhsExponent, RhsRadix>> & rhs);

// for each binary arithmetic, comparison, logic and bitwise operator, @
template <class LhsRep, int LhsExponent, int LhsRadix, class RhsRep, int RhsExponent, int RhsRadix>
constexpr auto operator@(
    const scaled_integer<LhsRep, power<LhsExponent, LhsRadix>> & lhs,
    const scaled_integer<RhsRep, power<RhsExponent, RhsRadix>> & rhs);

template <class LhsRep, int LhsExponent, int LhsRadix, class RhsFloat,
    typename = _impl::enable_if_t<numeric_limits<RhsFloat>::is_iec559>>
constexpr auto operator@(
    const scaled_integer<LhsRep, power<LhsExponent, LhsRadix>> & lhs,
    const RhsFloat & rhs);
template <class LhsFloat, class RhsRep, int RhsExponent, int RhsRadix,
    typename = _impl::enable_if_t<numeric_limits<LhsFloat>::is_iec559>>
constexpr auto operator@(
    const LhsFloat & lhs,
    const scaled_integer<RhsRep, power<RhsExponent, RhsRadix>> & rhs);

template <class LhsRep, int LhsExponent, int LhsRadix, class RhsInteger,
    typename = _impl::enable_if_t<numeric_limits<RhsInteger>::is_integer>>
constexpr auto operator@(
    const scaled_integer<LhsRep, power<LhsExponent, LhsRadix>> & lhs,
    const RhsInteger & rhs);
template <class LhsInteger, class RhsRep, int RhsExponent, int RhsRadix,
    typename = _impl::enable_if_t<numeric_limits<LhsInteger>::is_integer>>
constexpr auto operator@(
    const LhsInteger & lhs,
    const scaled_integer<RhsRep, power<RhsExponent, RhsRadix>> & rhs);

template <class LhsRep, int LhsExponent, int LhsRadix, auto RhsValue>
constexpr auto operator@(
    const scaled_integer<LhsRep, power<LhsExponent, LhsRadix>> & lhs,
    constant<RhsValue>);
template <auto LhsValue, class RhsRep, int RhsExponent, int RhsRadix>
constexpr auto operator@(
    constant<LhsValue>,
    const scaled_integer<RhsRep, power<RhsExponent, RhsRadix>> & rhs);

// for each arithmetic, comparison, logic and bitwise compound assignment operator, @=
template <class LhsRep, int LhsExponent, int LhsRadix, class RhsRep, int RhsExponent, int RhsRadix>
constexpr auto operator@=(
    scaled_integer<LhsRep, power<LhsExponent, LhsRadix>> & lhs,
    const scaled_integer<RhsRep, power<RhsExponent, RhsRadix>> & rhs);

template <class LhsRep, int LhsExponent, int LhsRadix, class RhsFloat,
    typename = _impl::enable_if_t<numeric_limits<RhsFloat>::is_iec559>>
constexpr auto operator@=(
    scaled_integer<LhsRep, power<LhsExponent, LhsRadix>> & lhs,
    const RhsFloat & rhs);
template <class LhsFloat, class RhsRep, int RhsExponent, int RhsRadix,
    typename = _impl::enable_if_t<numeric_limits<LhsFloat>::is_iec559>>
constexpr auto operator@=(
    LhsFloat & lhs,
    const scaled_integer<RhsRep, power<RhsExponent, RhsRadix>> & rhs);

template <class LhsRep, int LhsExponent, int LhsRadix, class RhsInteger,
    typename = _impl::enable_if_t<numeric_limits<RhsInteger>::is_integer>>
constexpr auto operator@=(

```

```

        scaled_integer<LhsRep, power<LhsExponent, LhsRadix>> & lhs,
        const RhsInteger & rhs);
template <class LhsInteger, class RhsRep, int RhsExponent, int RhsRadix,
        typename = _impl::enable_if_t<numeric_limits<LhsInteger>::is_integer>
        constexpr auto operator@=(
        LhsInteger & lhs,
        const scaled_integer<RhsRep, power<RhsExponent, RhsRadix>> & rhs);

template <class LhsRep, int LhsExponent, int LhsRadix, auto RhsValue>
        constexpr auto operator@=(
        scaled_integer<LhsRep, power<LhsExponent, LhsRadix>> & lhs,
        constant<RhsValue>);

template <auto Value>
scaled_integer(::cnl::constant<Value>)
-> /* ... */;

template <class Integer>
scaled_integer(Integer)
-> scaled_integer<Integer, 0>;
}

```

scaled_integer<> Class Template

```

template <int Exponent = 0, int Radix = 2>
struct power {};

template <class Rep = int, class Scale = power<>>
class scaled_integer
{
public:
    using rep = Rep;
    using radix = Radix;

    constexpr static int exponent;

    constexpr scaled_integer();
    template<class FromRep, int FromExponent, int FromRadix>
        constexpr scaled_integer(scaled_integer<FromRep, power<FromExponent, FromRadix>> const&);
    template<CNL_IMPL_CONSTANT_VALUE_TYPE Value>
        constexpr scaled_integer(constant<Value>);
    template<class S, _impl::enable_if_t<numeric_limits<S>::is_integer, int> Dummy = 0>
        constexpr scaled_integer(S const&);
    template<class S, _impl::enable_if_t<numeric_limits<S>::is_iec559, int> Dummy = 0>
        constexpr scaled_integer(S);
    template<class Numerator, class Denominator>
        constexpr scaled_integer(const fractional<Numerator, Denominator>&);
    template<class S, _impl::enable_if_t<numeric_limits<S>::is_integer, int> Dummy = 0>
        constexpr scaled_integer& operator=(S);
    template<class S, _impl::enable_if_t<numeric_limits<S>::is_iec559, int> Dummy = 0>
        constexpr scaled_integer& operator=(S);
    template<class FromRep, int FromExponent, int FromRadix>
        constexpr scaled_integer& operator=(
            scaled_integer<FromRep, power<FromExponent, FromRadix>> const&);
    template<class S, _impl::enable_if_t<numeric_limits<S>::is_integer, int> Dummy = 0>
        explicit constexpr operator S() const;
    template<class S, _impl::enable_if_t<numeric_limits<S>::is_iec559, int> Dummy = 0>
        explicit constexpr operator S() const;
};

```

Open Issues

Library Support

Because the aim is to provide an alternative to existing arithmetic types which are supported by the standard library, it is conceivable that a future proposal might specialize existing class templates and overload existing functions.

Possible candidates for overloading include the functions defined in `<cmath>` and a templated specialization of `numeric_limits`. A new type trait, `is_scaled_integer`, would also be useful.

While `scaled_integer` is intended to provide drop-in replacements to existing built-ins, it may be preferable to deviate slightly from the behavior of certain standard functions. For example, overloads of functions from `<cmath>` will be considerably less concise, efficient and versatile if they obey rules surrounding error cases. In particular, the guarantee of setting `errno` in the case of an error prevents a function from being defined as pure. This highlights a wider issue surrounding the adoption of the functional approach and compile-time computation that is beyond the scope of this document.

One suggested addition is a specialization of `std::complex`. This would take the form:

```
template<class Rep, int Exponent, int Radix>
class complex<scaled_integer<Rep, power<Exponent, Radix>>>;
```

This type's arithmetic operators would differ from existing specializations because `scaled_integer<>` operators often return results of a different type to their operands. Hence signatures such as

```
template<class T>
complex<T> operator*( const complex<T>& lhs, const complex<T>& rhs);
```

would need to be replaced with:

```
template<class T>
auto operator*( const complex<T>& lhs, const complex<T>& rhs);
```

Extended Comparison Range

Comparison operations between two `scaled_integer` operands require that they both have the same exponent. When they do not, conversion takes place to ensure they do. Unfortunately, if the difference in exponents is too great, the conversion may cause an out-of-bounds condition.

However, where two operands have bits whose values are in ranges that do not overlap, it may not be necessary to perform a conversion which results in out-of-range results: a result that ensures they continue to not overlap may be sufficient. For example,

```
static_assert(scaled_integer<uint8_t>{0} < scaled_integer<uint8_t, power<128>>{4.e38});
```

requires that the right-hand operand be converted to `scaled_integer<uint8_t>`. This will result in the underlying integer being scaled up by 1000 bits, resulting in undefined behavior and/or a flushed value. But in this case, it only needed to be scaled by 8 bits in order for none of its bit values to overlap with those of the left-hand operand.

Allow Binary Operations if Radixes are Different

Does it make sense to allow binary operations which take, say, a base-2 and a base-10 number? The answer is relatively straightforward when one considers that all base-2 numbers can be expressed using base-10 numbers. What about a base-2 and a base-3 number? At this point, we may need to convert them to a base-6 number to proceed.

Next, how do the exponents interact in situations when the radixes are different? For example, when adding `scaled_integer<int, power<-2, 2>>` and `scaled_integer<int, power<-1, 4>>`, is the result the former or the latter? They are computationally equivalent because they both represent units of 0.25.

The likely solution is to choose a result type with the minimum radix and then the minimum exponent necessary in order to be able to represent all possible values. However, this may result in a set of operators which are surprising to the user. Thus is it tempting to simply forbid inter-radix operations. (Note: a similar problem is faced by `common_type(chrono::duration)`.)

Prior Art

Many examples of fixed-point support in programming languages and their libraries exist. While almost all of them aim for low run-time cost and expressive alternatives to raw integer manipulation, they vary greatly in detail and in terms of their interface.

One especially interesting dichotomy is between solutions which offer a discrete selection of fixed-point types and libraries which contain a continuous range of exponents through type parameterization.

N1169

One example of the former is found in proposal N1169 [N1169], the intent of which is to expose features found in certain embedded hardware. It introduces a succinct set of language-level fixed-point types and impose constraints on the number of integer or fractional digits each can possess.

As with all examples of discrete-type fixed-point support, the limited choice of exponents is a considerable restriction on the versatility and expressiveness of the API.

Nevertheless, it may be possible to harness performance gains provided by N1169 fixed-point types through explicit template specialization. This is likely to be a valuable proposition to potential users of the library who find themselves targeting platforms which support fixed-point arithmetic at the hardware level.

P0106

There are many other C++ libraries available which fall into the latter category of continuous-range fixed-point arithmetic [mizvekov] [schregle] [viboos]. In particular, an existing library proposal by Lawrence Crowl, P0106 [P0106] (formerly N3352), aims to achieve very similar goals through similar means and warrants closer comparison than N1169.

P0106 introduces four class templates covering the quadrant of signed versus unsigned and fractional versus integer numeric types. It is intended to replace built-in types in a wide variety of situations and accordingly, is highly compile-time configurable in terms of how rounding and overflow are handled. Parameters to these four class templates include the range in bits and - for fractional types - the resolution.

The `scaled_integer` class template could probably - with a few caveats - be generated using the two fractional types, `nonnegative` and `negatable`, replacing the `Rep` parameter with the integer bit count of `Rep`, specifying `fastest` for the rounding mode and specifying `undefined` as the overflow mode.

However, `scaled_integer` more closely and concisely caters to the needs of users who already use integer types and simply desire a less error-prone form. It more closely follows the five design aims of this paper and, arguably, more closely follows the spirit of the standard in its pursuit of zero-cost abstraction.

Some aspects of the design of the P0106 API which back up these conclusion are that:

- the nature of the range-specifying template parameters - through careful framing in mathematical terms - abstracts away valuable information regarding machine-critical type size information;
- the breaking up of duties amongst four separate class templates introduces four new concepts and incurs additional mental load for relatively little gain while further detaching the interface from vital machine-level details;
- the absence of the most negative number from signed types reduces the capacity of all types by one and
- the selection of rounding and overflow modes via enumerations limits the choices to a pre-existing finite set and prevents the user from providing their own such strategies — or indeed from customizing fixed-point types in more exotic ways such as by using non-standard, platform-specific integer types.

A more detailed comparison of the approaches taken in this paper and P0106 can be found in [P0554].

Ada Language Support

Most languages lack fixed-point support. One difficulty in supporting fixed-point in a type system is that applications of fixed-point are sensitive to the choice of exponent value and parameterization of types is limited by syntax.

Ada provides binary and decimal fixed-point types and specifies resolution and range using `delta` and `range` literals, e.g.:

```
type T is delta 0.0625 range 0.0 .. 16.0;
```

Limitations imposed on C++'s UDL syntax and non-type template parameters make it difficult to form types using literals in this way.

Acknowledgements

SG6: Davis Herring, Lawrence Crowl, Lisa Lippincott

SG14: Guy Davidson, Michael Wong

Contributors: Ed Ainsley, Billy Baker, Lance Dyson, Marco Foco, Mathias Gaunard, Clément Grégoire, Nicolas Guillemot, Kurt Guntheroth, Matt Kinzelman, Joël Lamotte, Sean Middleditch, Paul Robinson, Patrice Roy, Peter Schregle, Ryhor Spivak

Revisions

This paper revises [P0037R6](#):

- rename `fixed_point` to `scaled_integer` following feedback from SG6 and LEWG in Kona
- replaced `Exponent` and `Radix` parameters with the `power` tag type
- remove sections discussing `fixed_point` versus `scaled_integer` including:
 - division operator issues
 - details of template parameters
- added mention of fixed-point types in Ada

P0037R6 revises [P0037R5](#):

- response to [\[P1368\]](#)
 - added section, [Division Operator](#)
 - added section, [Rename `fixed\_point` to avoid confusion over division](#)
 - added detail to section, [Custom Division](#)

P0037R5 revises [P0037R4](#):

- feedback from SG6 chair:
 - removed `multiply` and `divide` functions
 - added `Radix` non-type template parameter to `fixed_point`
 - added discussion of operators with different radixes
 - removed discussion of operators with different exponents
 - put case for rearranging template parameters of `fixed_point` to put `Exponent` first
 - corrected innacurate or out-of-date information in the P0106 prior art section
- added references to `fractional` and `elastic_integer`
- added deduction guides and `fractional` c'tor and assignment operator

P0037R4 revises [P0037R3](#):

- removed mention of `width` and `set_width`
- rewritten description of `Rep` template parameter
- added sections, **Access to Rep Value** and **Class Template Deduction**
- removed `make_fixed` and `make_ufixed` function templates
- rewritten **Operator Overloads** section, renamed **Operators** and included `constant` operators
- removed sections, **Overflow** and **Underflow**
- removed `add` and `subtract` function templates
- reversed the roles of `operator/` and `divide`
- replaced section on composability with link to [\[P0554\]](#)
- replaced reference to [\[P0381\]](#) with reference to [\[P0675\]](#)
- revised synopsis
- renamed section **Future Issues** to **Open Issues** and added sections,
 - **Different Division Strategies, Template Parameter Order, Disable Addition/Subtraction if Exponents are Different, Named Functions, Extended Comparison Range** and remove sections
 - **Compile-Time Bit-Shift Operations, Alternative Return Type Policies,**
- removed section, **References**, moving links into document body
- rewrote **Appendix 1: Reference Implementation** referencing [\[github\]](#)
- formatting changes intended to make markdown more readable as plain text

Appendix 1: Reference Implementation

An in-development implementation of the `fixed_point` class template and its essential supporting functions and types is available [\[github\]](#).

Appendix 2: Performance

Despite a focus on usable interface and direct translation from integer-based fixed-point operations, there is an overwhelming expectation that the source code result in minimal instructions and clock cycles. A few preliminary numbers are presented to give a very early idea of how the API might perform.

Some notes:

- A few test functions were run, ranging from single arithmetic operations to basic geometric functions, performed against integer, floating-point and fixed-point types for comparison.
- Figures were taken from a single CPU, OS and compiler, namely:
 - Debian clang version 3.5.0-10 (tags/RELEASE_350/final) (based on LLVM 3.5.0)
 - Target: x86_64-pc-linux-gnu
 - Thread model: posix
- Fixed inputs were provided to each function, meaning that branch prediction rarely fails. Results may also not represent the full range of inputs.
- Details of the test harness used can be found in the source project mentioned in Appendix 1;
- Times are in nanoseconds;
- Code has not yet been optimized for performance.

Types

Where applicable various combinations of integer, floating-point and fixed-point types were tested with the following identifiers:

- `uint8_t`, `int8_t`, `uint16_t`, `int16_t`, `uint32_t`, `int32_t`, `uint64_t` and `int64_t` built-in integer types;
- `float`, `double` and `long double` built-in floating-point types;
- `s3:4`, `u4:4`, `s7:8`, `u8:8`, `s15:16`, `u16:16`, `s31:32` and `u32:32` format fixed-point types.

Basic Arithmetic

Plus, minus, multiplication and division were tested in isolation using a number of different numeric types with the following results:

```
name cpu_time
add(float) 1.78011
add(double) 1.73966
add(long double) 3.46011
add(u4_4) 1.87726
add(s3_4) 1.85051
add(u8_8) 1.85417
add(s7_8) 1.82057
add(u16_16) 1.94194
add(s15_16) 1.93463
add(u32_32) 1.94674
add(s31_32) 1.94446
add(int8_t) 2.14857
add(uint8_t) 2.12571
add(int16_t) 1.9936
add(uint16_t) 1.88229
add(int32_t) 1.82126
add(uint32_t) 1.76
add(int64_t) 1.76
add(uint64_t) 1.83223
sub(float) 1.96617
sub(double) 1.98491
```

```

sub(long double) 3.55474
sub(u4_4) 1.77006
sub(s3_4) 1.72983
sub(u8_8) 1.72983
sub(s7_8) 1.72983
sub(u16_16) 1.73966
sub(s15_16) 1.85051
sub(u32_32) 1.88229
sub(s31_32) 1.87063
sub(int8_t) 1.76
sub(uint8_t) 1.74994
sub(int16_t) 1.82126
sub(uint16_t) 1.83794
sub(int32_t) 1.89074
sub(uint32_t) 1.85417
sub(int64_t) 1.83703
sub(uint64_t) 2.04914
mul(float) 1.9376
mul(double) 1.93097
mul(long double) 102.446
mul(u4_4) 2.46583
mul(s3_4) 2.09189
mul(u8_8) 2.08
mul(s7_8) 2.18697
mul(u16_16) 2.12571
mul(s15_16) 2.10789
mul(u32_32) 2.10789
mul(s31_32) 2.10789
mul(int8_t) 1.76
mul(uint8_t) 1.78011
mul(int16_t) 1.8432
mul(uint16_t) 1.76914
mul(int32_t) 1.78011
mul(uint32_t) 2.19086
mul(int64_t) 1.7696
mul(uint64_t) 1.79017
div(float) 5.12
div(double) 7.64343
div(long double) 8.304
div(u4_4) 3.82171
div(s3_4) 3.82171
div(u8_8) 3.84
div(s7_8) 3.8
div(u16_16) 9.152
div(s15_16) 11.232
div(u32_32) 30.8434
div(s31_32) 34
div(int8_t) 3.82171
div(uint8_t) 3.82171
div(int16_t) 3.8
div(uint16_t) 3.82171
div(int32_t) 3.82171
div(uint32_t) 3.81806
div(int64_t) 10.2286
div(uint64_t) 8.304

```

Among the slowest types are `long double`. It is likely that they are emulated in software. The next slowest operations are fixed-point multiply and divide operations - especially with 64-bit types. This is because values need to be promoted temporarily to double-width types. This is a known fixed-point technique which inevitably experiences slowdown where a 128-bit type is required on a 64-

bit system.

Here is a section of the disassembly of the s15:16 multiply call:

```
30:  mov    %r14,%rax
      mov    %r15,%rax
      movslq -0x28(%rbp),%rax
      movslq -0x30(%rbp),%rcx
      imul   %rax,%rcx
      shr    $0x10,%rcx
      mov    %ecx,-0x38(%rbp)
      mov    %r12,%rax
4c:  movzbl  (%rbx),%eax
      cmp    $0x1,%eax
      ↓ jne    68
54:  mov     0x8(%rbx),%rax
      lea    0x1(%rax),%rcx
      mov    %rcx,0x8(%rbx)
      cmp    0x38(%rbx),%rax
      ↑ jnb    30
```

The two 32-bit numbers are multiplied together and the result shifted down - much as it would if raw `int` values were used. The efficiency of this operation varies with the exponent. An exponent of zero should mean no shift at all.

3-Dimensional Magnitude Squared

A fast `sqrt` implementation has not yet been tested with `fixed_point`. (The naive implementation takes over 300ns.) For this reason, a magnitude-squared function is measured, combining multiply and add operations:

```
template <class FP>
constexpr FP magnitude_squared(const FP & x, const FP & y, const FP & z)
{
    return x * x + y * y + z * z;
}
```

Only real number formats are tested:

```
float 2.42606
double 2.08
long double 4.5056
s3_4 2.768
s7_8 2.77577
s15_16 2.752
s31_32 4.10331
```

Again, the size of the type seems to have the largest impact.

Circle Intersection

A similar operation includes a comparison and branch:

```
template <class Real>
bool circle_intersect_generic(Real x1, Real y1, Real r1, Real x2, Real y2, Real r2)
{
    auto x_diff = x2 - x1;
    auto y_diff = y2 - y1;
    auto distance_squared = x_diff * x_diff + y_diff * y_diff;

    auto touch_distance = r1 + r2;
    auto touch_distance_squared = touch_distance * touch_distance;
```

```
    return distance_squared <= touch_distance_squared;  
}
```

float 3.46011

double 3.48

long double 6.4

s3_4 3.88

s7_8 4.5312

s15_16 3.82171

s31_32 5.92

Again, fixed-point and native performance are comparable.